In [1]:
```
from IPython.display import display, HTML

# 💡 ONLY SET YOUR TIME HERE (IN MINUTES)
EXAM_MINUTES = 45

html_code = f"""
<div style="max-width: 400px; padding: 15px; border: 1px solid #ccc; border-
    <div style="font-size: 16px; font-weight: bold; margin-bottom: 8px; colo
    <div id="exam-timer" style="font-size: 36px; font-family: monospace; for
    <div style="width: 100%; background-color: #e0e0e0; border-radius: 4px;
        <div id="exam-progress" style="width: 100%; height: 100%; backgrounc
    </div>
    <button id="timer-control-btn" style="padding: 8px 20px; font-size: 14px
</div>

<script>
(() => {{
    const TOTAL_MINUTES = {EXAM_MINUTES};
    let totalSeconds = TOTAL_MINUTES * 60;
    const initialSeconds = totalSeconds;
    let isRunning = false;

    const display = document.getElementById('exam-timer');
    const progress = document.getElementById('exam-progress');
    const controlBtn = document.getElementById('timer-control-btn');

    if (window.examTimerInterval) clearInterval(window.examTimerInterval);

    function updateTimerVisuals() {{
        if (totalSeconds <= 0) {{
            clearInterval(window.examTimerInterval);
            display.innerText = "TIME'S UP!";
            display.style.color = "#c0392b";
            progress.style.width = "0%";
            controlBtn.style.display = "none";
            return;
        }}

        const mins = String(Math.floor(totalSeconds / 60)).padStart(2, '0');
        const secs = String(totalSeconds % 60).padStart(2, '0');
        display.innerText = mins + ":" + secs;

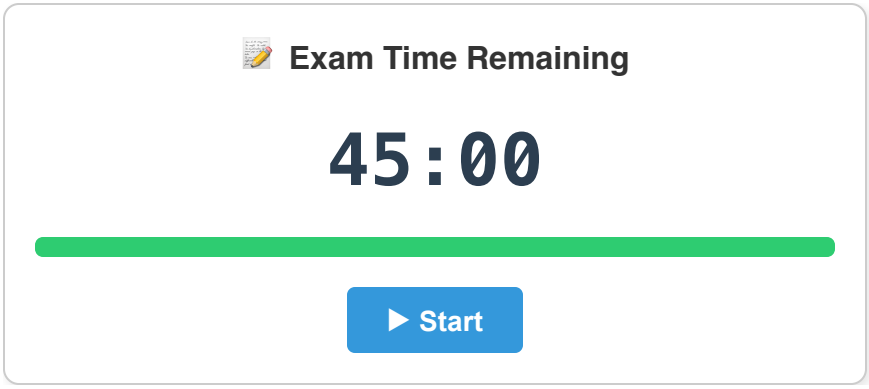
        const percentage = (totalSeconds / initialSeconds) * 100;
        progress.style.width = percentage + "%";

        if (totalSeconds <= 300) {{
            display.style.color = "#c0392b";
            progress.style.backgroundColor = "#e74c3c";
        }} else if (totalSeconds <= 900) {{
            display.style.color = "#d35400";
            progress.style.backgroundColor = "#f39c12";
        }}
    }}
```

```
    controlBtn.addEventListener('click', () => {{
        if (!isRunning) {{
            // Start the countdown
            isRunning = true;
            controlBtn.innerText = "⏸ Pause";
            controlBtn.style.backgroundColor = "#e67e22";

            window.examTimerInterval = setInterval(() => {{
                totalSeconds--;
                updateTimerVisuals();
            }}, 1000);
        }} else {{
            // Pause the countdown
            isRunning = false;
            clearInterval(window.examTimerInterval);
            controlBtn.innerText = "▶ Resume";
            controlBtn.style.backgroundColor = "#2ecc71";
        }}
    }});
}})();
</script>
"""

display(HTML(html_code))
```



DATA1030 Exam 2

Start time: 10:30am

End time: 11:15am

Please leave one empty seat between students and spread out as much as possible.

When you are done, take a picture of your document and submit it to Gradescope to the "second exam" assignment. I'll submit the grades there.

After the grades are released, I'll announce a drop-in session if you'd like to discuss your answers. You can also submit a regrade request on Gradescope if you'd like a more detailed feedback.

Mudcard

- **I still don't unsderstand what sampling from a dataset for a decision tree mean.**
  - Decision trees don't train on the whole training set.
  - They train on a subset (a sample) of the points and a subset of the features.
  - So if you have 100 rows (samples) and 10 columns (features) in a training set, a decision tree might only see 60 randomly selected points and 5 randomly selected features

A note on imbalanced datasets

- we learnt that a classification problem is imbalanced if more than 90-95% of the points belong to one class (class 0) and only a small fraction of the points belong to the other class (class 1)
  - fraud detection
  - sick or not sick (usually by far most people are not sick)
- we learnt to not use a metric that relies on the True Negatives in the confusion matrix
  - no accuracy or ROC
  - use f_beta or the precision-recall curve instead

What else can I do if I have an imbalanced dataset?

- most (but not all) classification algorithms we covered have a parameter called `class_weight` which allows you to assign more weight to the class 1 point
  - a misclassified class 1 point will contribute more to the cost function than a misclassified class 0 point
  - read the manual on `class_weight` because the different algorithms have slightly different definitions for this parameter
  - usually you can use `None`, `balanced`, or manually define what the class weight should be
  - it is worthwhile to tune this parameter if you have an imbalanced dataset
- resample/augment the dataset
  - SMOTE (Synthetic Minority Over-sampling Technique), see the paper

- to improve the balance of the problem, new class 1 examples are synthesized from the existing examples
- be careful though!
  - while resampling improves the balance of the dataset, the results of the model can be misleading
  - when you deploy the model, the incoming data will be as imbalanced as the original data

Misleading results with resampling

- let's assume you have an imbalanced dataset with 99% of points in class 0 and 1% of points in class 1
- you resample it such that the improved class balance is 50-50
- here the confusion matrix of the trained model:

		Predicted class	
		Predicted Negative (0)	Predicted Positive (1)
Actual class	Condition Negative (0)	**True Negative (TN): 45%**	**False Positive (FP): 5%**
	Condition Positive (1)	**False Negative (FN): 5%**	**True Positive (TP): 45%**

- 90% accuracy which is well above the 50% baseline accuracy!
- the precision, recall, and f1 scores are all 0.9.
- it looks great, doesn't it?
- let's rewrite the confusion matrix to reflect rates with respect to the Condition Negative and Condition Positive points!

		Predicted class	
		Predicted Negative (0)	Predicted Positive (1)
Actual class	Condition Negative (0) 50% of the points	**90% of CNs are correctly classified**	**10% of CNs are incorrectly classified**
	Condition Positive (1) 50% of the points	**10% of CPs are incorrectly classified**	**90% of CPs are correctly classified**

Let's deploy this model

- the incoming data has the same balance as the original dataset (99% to 1%)
- let's assume we have 1e5 new points, 9.9e4 belongs to class 0, 1000 belongs to class 1
- what will be the numbers in the confusion matrix?

		Predicted class	
		Predicted Negative (0)	Predicted Positive (1)
Actual class	Condition Negative (0) 99000 points	**True Negative (TN): 99000 * 0.9 = 89100**	**False Positive (FP): 99000 * 0.1 = 9900**
	Condition Positive (1) 1000 points	**False Negative (FN): 1000 * 0.1 = 100**	**True Positive (TP): 1000 * 0.9 = 900**

- the accuracy of this model is still 0.90 but now it is well below the baseline of 0.99!
- recall is good (0.90) but the precision is not great (~0.083)
- the f1 score is ~0.15
- the false positives are overwhelming
- this is why you need to be careful with resampling

Quiz

Mudcard

In [ ]: